



Slide 1
CS 170
Java Programming 1
Arrays and Loops
 Duration: 00:01:27
 Advance mode: Auto

Notes:

Welcome to the CS170, Java Programming 1 lecture on Arrays and Loops.

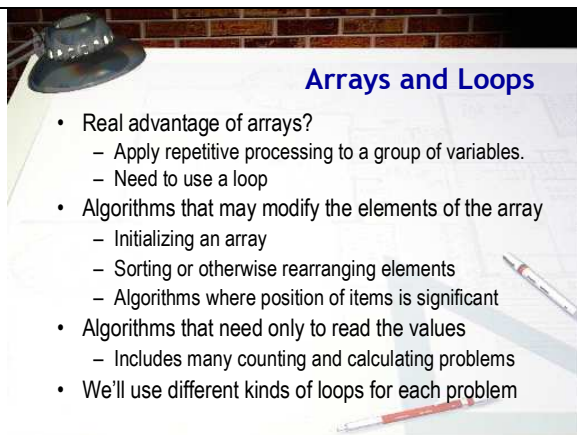
Loop Guru, the album by Duniya, features "Nearly 69 minutes of infectious electronic rhythms ranging from wirly dance grooves to worldly atmospheric shivers that take us from the dance floor to the chill room".

Whatever; I just like the name of the album.

Instead of wirly dance grooves, in this lecture we'll take a look at various ways that we can process arrays, many of them designed to turn you into a real loop guru. We'll look at:

- Algorithms that process and modify the elements of an entire array, such as initialization.
- Read-only algorithms that can make use of Java's foreach loop syntax for visiting each array element.
- Algorithms that process a portion of an array by using sentinel loops.
- Algorithms that modify a portion of an array by inserting or deleting elements.

We'll pick up many of these topics next week. For right now, let's start by learning about the algorithms that process an entire array, and that may, (or may not) modify the array elements. This includes operations like initialization, sorting, and reversing the elements in an array, (something you'll need to do for the exercises).



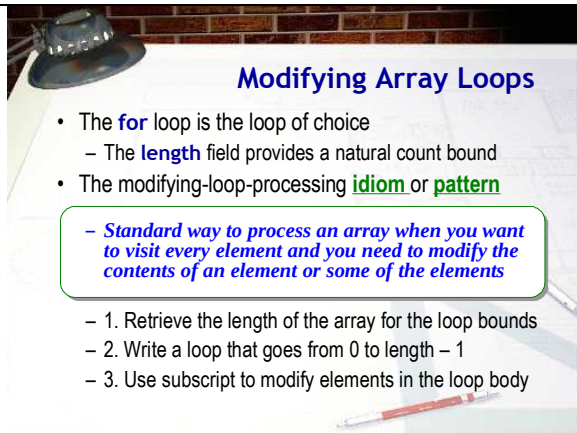
Slide 2
Arrays and Loops
 Duration: 00:00:53
 Advance mode: Auto

Notes:

The real advantage of arrays, is that they allow you to apply the same processing to all of the elements by using a loop. We can divide the processing of entire arrays into two kinds:

1. Algorithms that may modify the elements of the array as they are processed. This includes initialization, sorting, and otherwise rearranging items. It also includes algorithms where the position of the elements in the array is significant or must be noted.
2. Algorithms that need only to read the values contained in the array. These algorithms solve many counting and calculating problems.

We'll use different kinds of loops for both types of algorithms, although you can use the modifying for loop on the read-only calculation problems as well.



Modifying Array Loops

- The **for** loop is the loop of choice
 - The **length** field provides a natural count bound
- The modifying-loop-processing **idiom** or **pattern**
 - Standard way to process an array when you want to visit every element and you need to modify the contents of an element or some of the elements*

- Retrieve the length of the array for the loop bounds
- Write a loop that goes from 0 to length - 1
- Use subscript to modify elements in the loop body

Slide 3 

Modifying Array Loops

Duration: 00:01:27
Advance mode: Auto

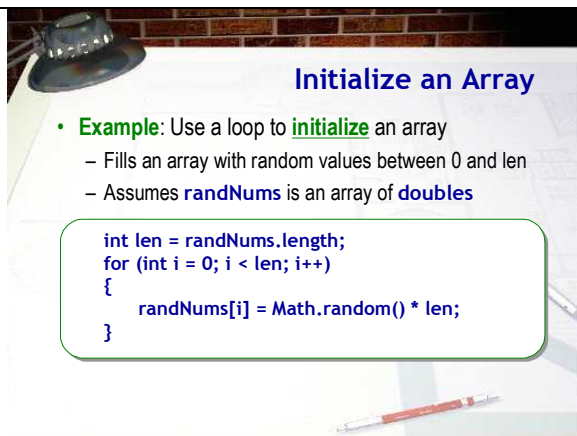
Notes:

For array processing, where the elements may be modified, or where the positions of the elements is important, the for loop is the loop of choice, because the array.length field creates a natural counter that you can use to control the loop bounds, and because the loop index can be used as the subscript to access the array elements.

To do this, you'll follow the modifying-loop-processing idiom or pattern. A pattern is like a recipe you use whenever you encounter a particular type of problem. The modifying-loop-processing pattern is the standard way to process an array, when you want to visit every element, and, when you need to modify one or more of the elements. If you need to initialize an array to a value other than zero, for instance, the initializer list is useful only for short arrays. Typing in even 25 or 50 values can get very tedious. Instead, you should use a for loop that follows this pattern:

- Retrieve the length of the array and use it as the loop's upper bounds.
- Write a for loop that goes from 0 to less-than the upper bounds.
- Assign a new value to each element that you wish to modify inside the loop body. Use the loop counter as the element's subscript.

Let me show you an example, and then you can try a few on your own.



Initialize an Array

- Example:** Use a loop to **initialize** an array
 - Fills an array with random values between 0 and len
 - Assumes **randNums** is an array of **doubles**

```
int len = randNums.length;
for (int i = 0; i < len; i++)
{
    randNums[i] = Math.random() * len;
}
```

Slide 4 

Initialize an Array

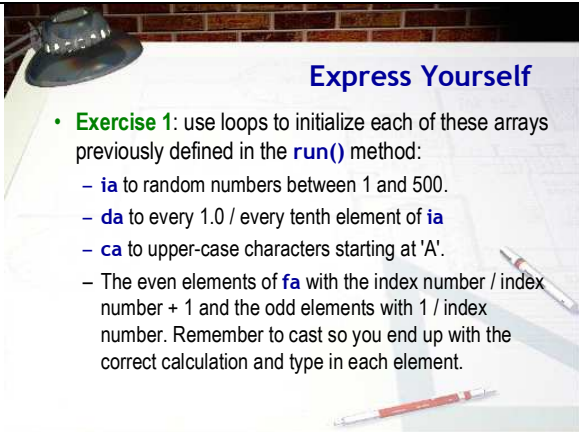
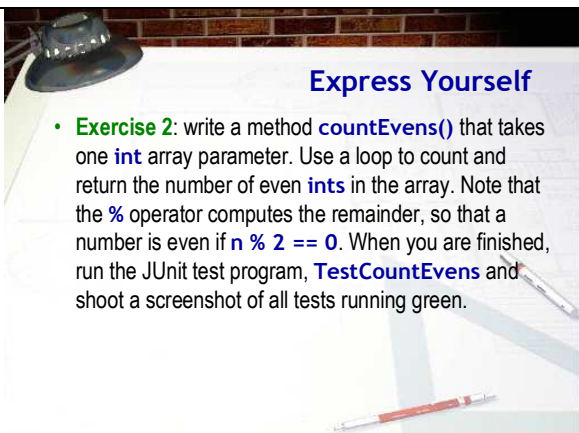
Duration: 00:01:17
Advance mode: Auto

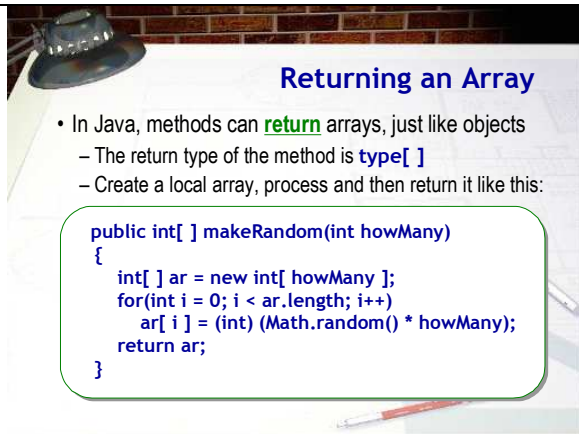
Notes:

Here's an example that uses this pattern to initialize an array. In this case, I'm going to fill each element in an array of doubles (randNum) with random values that fall somewhere between 0 and less than the length of the array. (If the length of the array was 10, the values could be anything from 0.0 to 9.99999).

Notice the way that the code follows the recipe or pattern:

- Retrieve the length of the array and use it as the loop's upper bounds.
- Write a for loop that goes from 0 to less-than the upper bounds. Notice that when processing an array (or a String), you'll always want to use asymmetric bounds—that is, bounds that start at 0 and go to less than the upper bound—since both Strings and arrays are 0 based and the last element is always at len - 1. You can read more about symmetric and asymmetric bounds in Quality Tip 6.3 on page 252 in your text.
- Finally, we assign a new value to each element that we wish to modify inside the loop body, using the loop

	counter as the element's subscript.
 Express Yourself • Exercise 1: use loops to initialize each of these arrays previously defined in the <code>run()</code> method: – <code>ia</code> to random numbers between 1 and 500. – <code>da</code> to every 1.0 / every tenth element of <code>ia</code> – <code>ca</code> to upper-case characters starting at 'A'. – The even elements of <code>fa</code> with the index number / index number + 1 and the odd elements with 1 / index number. Remember to cast so you end up with the correct calculation and type in each element. Slide 5 Express Yourself Duration: 00:01:50 Advance mode: Auto	Notes: OK. Now it's your turn. Open Eclipse and the <code>ArrayTest</code> class. Inside the <code>run()</code> method, initialize each of these arrays that you previously defined in the <code>run()</code> method: • Initialize <code>ia</code> to hold random numbers between 1 and 500. • Initialize <code>da</code> so that each element contains 1.0/ every tenth element in the array <code>ia</code>. In other words, the first element will be 1.0/ the first element in <code>ia</code>. The second element in <code>da</code> will be 1.0/ the 10th element in <code>ia</code>. The third element in <code>da</code> will be 1.0/ the 20th element in <code>ia</code> and so on. • Initialize <code>ca</code> to the upper-case characters starting at capital 'A'. • For <code>fa</code>, initialize the even numbered elements (0, 2, 4, etc) with the subscript or index number divided by the index number + 1. Initialize the odd elements with 1 / index number. Remember to cast so that you Once you've finished, save your work (and make sure everything compiles with no red squiggles). Then, run the program just to make sure you don't have any runtime errors. When the console window shows no errors, snap a picture. Create a new section in your IC document this week, and paste your picture in as Exercise 1.
 Express Yourself • Exercise 2: write a method <code>countEvens()</code> that takes one <code>int</code> array parameter. Use a loop to count and return the number of even <code>ints</code> in the array. Note that the <code>%</code> operator computes the remainder, so that a number is even if <code>n % 2 == 0</code>. When you are finished, run the JUnit test program, <code>TestCountEvens</code> and shoot a screenshot of all tests running green. Slide 6 Express Yourself Duration: 00:00:57	Notes: Let's try another exercise that processes all of the elements in the loop. The only difference between this and the previous one is that in the loop body, you won't modify the array elements, but simply examine them. Inside the <code>ArrayTest</code> program, write a method named <code>countEvens()</code>. The <code>countEvens()</code> method will take one parameter: an array of <code>int</code>. Use a loop to count the number of even ints in the array. (Note that this means you want to count the number of array values that are even, not the number of array subscripts that are even.) A number is even if <code>n % 2 == 0</code>. When you have finished with the method, and <code>ArrayTest</code> compiles, then run the JUnit test program, <code>TestCountEvens</code>. Shoot me a screenshot for Exercise 2 showing both your code and the JUnit tests running all green.



Returning an Array

- In Java, methods can **return** arrays, just like objects
 - The return type of the method is **type[]**
 - Create a local array, process and then return it like this:

```
public int[] makeRandom(int howMany)
{
    int[] ar = new int[ howMany ];
    for(int i = 0; i < ar.length; i++)
        ar[ i ] = (int) (Math.random() * howMany);
    return ar;
}
```

Slide 7**Returning an Array**

Duration: 00:01:24

Advance mode: Auto

Notes:

In the last lecture you learned how to pass an array variable to a method where it could be processed. Java also allows you to easily write methods that return array variables. As with passing array variables to a method, the only "trick" is in knowing how to declare the return type.

Suppose, for instance, you wanted to write a method that returned an array of random integers. Your method header would look like this:

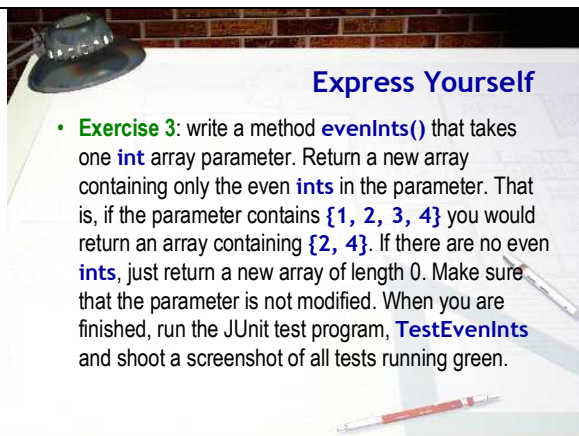
```
public int[] makeRandom(int howMany) {}
```

Notice that the return type is declared as `int[]` and not `int`.

Once you've figured out how to write the return type, the next question that arises is "How do you get the array variable to return?" The answer is simple--you create a local array variable inside your method, and then return the local variable.

Here's the rest of the `makeRandom()` method that shows you how. Notice that it:

1. Creates and initializes a local array variable using the `int` parameter `howMany` to determine the number of elements in the local array.
2. Uses the modifying-loop-processing pattern to visit every element in the local array
3. Fills each element with a random number
4. Returns the newly processed array



Express Yourself

- Exercise 3:** write a method `evenInts()` that takes one `int` array parameter. Return a new array containing only the even `ints` in the parameter. That is, if the parameter contains `{1, 2, 3, 4}` you would return an array containing `{2, 4}`. If there are no even `ints`, just return a new array of length 0. Make sure that the parameter is not modified. When you are finished, run the JUnit test program, `TestEvenInts` and shoot a screenshot of all tests running green.

Notes:

Let's finish up this week with an exercise that returns an array. Inside `ArrayTest`, add a new method, `evenInts` that takes one `int` array parameter. The method should also return an `int` array. Inside the method, create a new array containing only the `ints` with even values. For instance, if you supply an array containing the values 1, 2, 3, and 4, then the returned array would only contain 2 and 4—that is, the even integers. If there are no even integers in the input array, then return a new array of length 0.

Make sure when you write this program, that you don't modify the elements in the input array. To find the size of the

Slide 8

Express Yourself

Duration: 00:01:12

Advance mode: Auto

output array, you can call the `countEvens()` method that you wrote for Exercise 2, or you can simply use a second loop.

When you are finished, run the JUnit test program, `TestEvenInts` and shoot me a screenshot showing both your code and all of the tests running green.

Well, that's it for this week. See you next week.